

062-信号量 & 管程

信号量 & 管程

- 关中断/SWAP/TS/Peterson算法：反复测试
- 信号量/管程：单次测试

信号量与 PV 操作

```
// 整型信号量
typedef int semaphore_int;

// 可视为原子操作
void P (semaphore_int *s) {
    while (*s <= 0); // 等待信号量大于 0
    (*s)--; // 信号量减一
}

// 可视为原子操作
void V (semaphore_int *s) {
    (*s)++; // 信号量加一
}
```

```
// 记录型信号量
typedef struct {
    int value; // 信号量的值
    pcb_t *queue; // 等待队列
} Semaphore;

// 可视为原子操作
void P(Semaphore *s) {
    s->value--;
    if (s->value < 0)
        block(s->queue);
}

// 可视为原子操作
void V(Semaphore *s) {
    s->value++;
    if (s->value <= 0)
        wakeup(s->queue);
}

Semaphore mutex = {1, NULL};
void critical_section() {
    P(&mutex);
    // 临界区
    V(&mutex);
}
```

```
}
```

- 信号量 **s**：当前可用资源的数量，若为负则表示等待使用资源的进程数
- 整型信号量：不存在等待队列，会忙等
- 记录型信号量：有等待队列，若信号量小于 0，则阻塞进程
 - 每个资源/信号量对应一个等待队列
 - **信号量应该作为互斥使用，而非计数器，不得直接访问信号量的属性**
 - 如果希望实现原子的计数器，使用：信号量+变量
- 检测 **P**：运行态 -> 阻塞态
- 检测 **V**：阻塞态 -> 就绪态

用信号量操作进程

- 互斥：描述资源（椅子，盘子），不能同时访问
- 同步：描述存在“交互”的进程（理发师，柜员），需要保证某些操作的顺序
- 需要不断运行的进程要加上 **while(true)** 循环，单次运行的进程不需要

实现互斥

1. 划分临界区
2. 设置初始值为**1**的信号量**mutex**

```
Semaphore mutex = 1; // 做题的时候直接用数字初始化即可

P1(){
    P(&mutex); // 进入临界区
    // 临界区代码
    V(&mutex); // 离开临界区
}

P2(){
    P(&mutex); // 进入临界区
    // 临界区代码
    V(&mutex); // 离开临界区
}
```

实现同步

1. 分析同步关系，找出需要保证一前一后执行的代码
2. 设置初始值为**0**的信号量**sem**
3. 在前操作后执行**V(&sem)**
4. 在后操作前执行**P(&sem)**

实现互斥的**P**操作一定要在实现同步的**P**操作之后进行，否则会引发死锁

```

Semaphore sem = 0;

P1(){
    // 前置代码
    V(&sem); // 通知 P2
}

P2(){
    P(&sem); // 等待 P1
    // 后续代码
}

```

实现前驱 & 死锁

- 实现前驱：为每一对前驱关系实现同步（在出节点执行V，在入节点执行P）
- 死锁本质：循环等待
 - 信号量的前驱图中出现环路

管程

- 目的：封装复杂的 P/V 操作
- 组成
 - 共享数据结构（变量）
 - 对数据结构操作的过程（函数）
 - 初始化语句
 - 管程名称
- 特点
 - 进程只有调用管程内的过程，才能访问管程内的变量
 - 同一时刻只有一个进程在管程内执行管程内的某一个过程

```

TYPE philosopher_monitor = monitor
    semaphore forks[5];
    enum {THINKING, HUNGRY, EATING} state[5]; // 哲学家的状态
    for (int i = 0; i < 5; i++) {
        forks[i] = 1; // 初始化叉子为可用状态
        state[i] = THINKING; // 初始化哲学家状态为思考
    }
InterfaceModule IM;

DEFINE pick_forks, put_forks;
USE wait, signal, enter, leave;

void pickup(int i) {
    enter(IM); // 进入管程
    state[i] = hungry; // 表示哲学家想吃饭
    test(i); // 尝试进入吃饭状态
    if (state[i] != eating) // 如果没能成功吃饭
        wait(self[i], self_count[i], IM); // 等待条件变量
    leave(IM); // 离开管程
}

```

```

}

void putdown(int i) {
    enter(IM);
    state[i] = thinking;           // 放下筷子, 进入思考状态
    test((i - 1) % 5);             // 尝试唤醒左邻
    test((i + 1) % 5);             // 尝试唤醒右邻
    leave(IM);
}

void test(int k) {
    if ((state[(k - 1) % 5] != eating) && // 若左右哲学家都没有在吃饭, 则可以吃饭
        (state[k] == hungry) &&
        (state[(k + 1) % 5] != eating)) {
        state[k] = eating;
        signal(self[k], self_count[k], IM);
    }
}

```

- **condition**: 条件变量, 用于进程间的同步
- **wait**: 使当前进程阻塞, 直到条件满足
- **signal**: 唤醒一个等待该条件的进程, 若无等待进程则无效
 - 执行**signal**后, 执行者和被唤醒着都在管程内, 执行者应等待被唤醒进程退出管程/等待另一条件变量
- **wait signal** 对应硬件层面的原子操作, 由编译器实现

Hoare 管程

- **mutex**: 管程级别的互斥锁, 确保同一时刻只有一个进程在管程内执行
 - 进程调用管程过程时, 应执行 **P(mutex)**
 - 在管程结束后, 若有进程等待 (**next_count > 0**), 则执行 **V(next)** 唤醒一个等待的进程, 否则执行 **V(mutex)** 释放互斥锁
 - 执行**wait**时必须执行**V(mutex)**, 以确保其他进程可以进入管程
- **next next_count**: 记录管程内的等待进程
 - 发出**signal**时, 应使用**P(next)**阻塞自己
- **x_sem x_count**: 记录条件变量的等待进程
 - 执行**wait(x_sem)**时, 若**x_count > 0**, 则执行**P(x_sem)**阻塞自己
 - 执行**signal(x_sem)**时, 若**x_count > 0**, 则执行**V(x_sem)**唤醒一个等待的进程
- 实际应用中, 需要有全局的状态变量给程序员, 程序员判断状态是否符合条件, 如果不符合就**wait**

```

// 代码风格太糟糕了
typedef struct InterfaceModule {
    semaphore mutex; // 管程级别的互斥锁
    semaphore next;  // 管程内的等待进程
    int next_count;  // 管程内等待进程的数量
}

mutex = 1;
next = 0;

```

```
next_count = 0;

void enter(InterfaceModule &im)
{
    P(im->mutex);
}

void leave(InterfaceModule &im)
{
    if (im->next_count > 0) {
        V(im->next); // 唤醒一个等待的进程
    } else {
        V(im->mutex); // 释放互斥锁
    }
}

void wait(semaphore &x_sem, int &x_count, InterfaceModule &IM)
{
    x_count++; // 等资源进程个数加1, x_count初始化为0
    if (IM.next_count > 0) // 判断是否有发出过signal的进程
        V(IM.next); // 有就释放一个
    else
        V(IM.mutex); // 否则开放管程
    P(x_sem); // 等资源进程阻塞自己, x_sem初始化为0
    x_count--; // 等资源进程个数减1
}

void signal(semaphore &x_sem, int &x_count, InterfaceModule &IM)
{
    if (x_count > 0) { // 判断是否有等待资源的进程
        IM.next_count++; // 发出signal进程个数加1
        V(x_sem); // 释放一个等资源的进程
        P(IM.next); // 发出signal进程阻塞自己
        IM.next_count--; // 发出signal进程个数减1
    }
}
```